Part 1 — Minimax Search: Explanation and Python Code

Let's understand the concept first, calculate the values of all the nodes in your game tree, and then implement the Python code.

1. What is Minimax?

Minimax is an algorithm used in two-player games to choose the best possible move, assuming that both players play optimally.

For example, consider games such as chess, tic-tac-toe, and checkers.

There are two players:

- **MAX:** Wants to get the highest possible score.
- **MIN:** Wants to get the lowest possible score for MAX.

Both players make decisions to achieve their own objectives. MAX tries to maximize the score, while MIN tries to prevent MAX from getting a high score.

How does Minimax work?

Your game tree has four levels:

1. **Root node R (MAX):** MAX chooses between A, B, and C.
2. **Second level (MIN):** MIN chooses the lowest-valued option available under each of A, B, and C.
3. **Third level (MAX):** MAX chooses the highest-valued option under each node at the second level.
4. **Terminal nodes:** These contain the final utility values, such as 3, 5, 6, and 9.

The important idea is that **Minimax works backward from the terminal values to calculate the best decision at the root.**

We do not simply choose the largest number in the entire tree. We must follow the MAX and MIN rules at each level.

---

2. Understanding your game tree

Let's calculate the values step by step.

Step 1: Calculate the third-level MAX nodes

At this level, MAX always selects the **maximum** of its two children.

Under A:

- A1 has terminal values 3 and 5.
- A2 has terminal values 6 and 9.

Therefore:

$$
A1 = \max(3,5)=5
$$

$$
A2 = \max(6,9)=9
$$

Under B:

- B1 has terminal values 1 and 2.
- B2 has terminal values 0 and -1.

Therefore:

$$
B1 = \max(1,2)=2
$$

$$
B2 = \max(0,-1)=0
$$

Under C:

- C1 has terminal values 7 and 4.
- C2 has terminal values 5 and 6.

Therefore:

$$
C1 = \max(7,4)=7
$$

[
C2 = \max(5,6)=6
]

Here is the summary:

Node	Terminal values	Minimax value
A1	3, 5	5
A2	6, 9	9
B1	1, 2	2
B2	0, -1	0
C1	7, 4	7
C2	5, 6	6

Remember: These nodes represent MAX's decisions, so we take the maximum value at each one.

Step 2: Calculate A, B, and C

Now we move one level up, where the player is MIN.

MIN chooses the smaller value from each pair of children.

For node A:

[
A = \min(A1,A2)
]

[
A = \min(5,9)=5
]

For node B:

[
B = \min(B1,B2)
]

[
B = \min(2,0)=0

]

For node C:

[
C = \min(C1,C2)
]

[
C = \min(7,6)=6
]

Our results are:

Node	Child values	Minimax value
A	5, 9	**5**
B	2, 0	**0**
C	7, 6	**6**

Remember: These nodes represent MIN's decisions, so we take the minimum value at each one.

Step 3: Calculate the root R

Finally, we reach the root node R, where MAX must choose between A, B, and C.

We have calculated:

- A = 5
- B = 0
- C = 6

Since R belongs to MAX:

[
R = \max(A,B,C)
]

[
R = \max(5,0,6)=6
]

Therefore:

The optimal first move for MAX is C, and the minimax value of the root is 6.

Why?

Because MAX compares the values of all three possible moves and chooses the largest one:

[
6 > 5 > 0
]

Even though the terminal value 9 appears in the tree, MAX cannot guarantee that result. If MAX chooses A, MIN will select A1, whose value is 5, rather than A2, whose value is 9.

Similarly, if MAX chooses C, MIN selects C2 because its value is 6, which is lower than C1's value of 7.

Consequently, choosing C guarantees MAX a value of 6, assuming both players play optimally.

---

3. Implementing Minimax in Python

Now that we understand the calculations, let's implement the two required functions.

Function 1: `minimax(node, maximizing_player=True)`

This function calculates the minimax value of any node.

Its logic is simple:

1. If the node is a terminal node, return its utility value.
2. If it is MAX's turn, calculate the values of its children and return the maximum.
3. If it is MIN's turn, calculate the values of its children and return the minimum.

The function calls itself recursively until it reaches terminal nodes.

Function 2: `minimax_decision(root='R')`

This function finds the best first move for MAX.

It evaluates each child of the root using `minimax()` and selects the child with the highest minimax value.

If multiple children have the same value, it keeps the first child, as required by the question.

Complete Python code

Copy and run this code in your notebook:

```
GAME_TREE = {
'R': ['A', 'B', 'C'],
'A': ['A1', 'A2'],
'B': ['B1', 'B2'],
'C': ['C1', 'C2'],
'A1': ['A1a', 'A1b'],
'A2': ['A2a', 'A2b'],
'B1': ['B1a', 'B1b'],
'B2': ['B2a', 'B2b'],
'C1': ['C1a', 'C1b'],
'C2': ['C2a', 'C2b']
}

UTILITY = {
'A1a': 3, 'A1b': 5,
'A2a': 6, 'A2b': 9,
'B1a': 1, 'B1b': 2,
'B2a': 0, 'B2b': -1,
'C1a': 7, 'C1b': 4,
'C2a': 5, 'C2b': 6
}

def minimax(node, maximizing_player=True):
if node in UTILITY:
return UTILITY[node]

values = []

for child in GAME_TREE[node]:
    value = minimax(child, not maximizing_player)
    values.append(value)

if maximizing_player:
    return max(values)
else:
    return min(values)

def minimax_decision(root='R'):
best_child = None
best_value = float('-inf')
```

```
for child in GAME_TREE[root]:
    value = minimax(child, False)

    if value > best_value:
        best_value = value
        best_child = child

return best_child, best_value
```

move, value = minimax_decision('R')

print('Best root move:', move)
print('Minimax value :', value)

print('\nMinimax values of internal nodes:')

for node in ['A', 'B', 'C', 'R']:
print(node, ':', minimax(node, node in ['A', 'B', 'C', 'R']))

Expected output

```
Best root move: C
Minimax value : 6

Minimax values of internal nodes:
A : 5
B : 0
C : 6
R : 6
```

4. Task 1: Final answer table

You can fill in the table in your assignment as follows:

Node	Minimax value
A	5
B	0
C	6
R	6

5. Explanation to write in your assignment

Here is a ready-to-use explanation of the result.

The Minimax algorithm recursively evaluates the game tree from the terminal nodes to the root. At MAX nodes, it selects the maximum value, while at MIN nodes, it selects the minimum value.

For node A, the values of A1 and A2 are 5 and 9, respectively. Therefore, A = min(5, 9) = 5. For node B, the values of B1 and B2 are 2 and 0, so B = min(2, 0) = 0. For node C, the values of C1 and C2 are 7 and 6, so C = min(7, 6) = 6.

At the root R, MAX selects the largest of the three values: max(5, 0, 6) = 6. Therefore, the optimal first move for MAX is C, and the minimax value of the root is 6.

---

6. Important concepts to remember

Concept	Meaning
`GAME_TREE`	Stores the children of each node in the specified order.
`UTILITY`	Stores the final scores of the terminal nodes.
`minimax()`	Recursively calculates the value of a node.
`max(values)`	Selects the largest value when it is MAX's turn.
`min(values)`	Selects the smallest value when it is MIN's turn.
`minimax_decision()`	Chooses the best first move for MAX.
`float('-inf')`	Represents negative infinity, allowing any valid utility to become the initial best value.

One important detail: In `minimax_decision()`, we call `minimax(child, False)` because the root is a MAX node. Once MAX chooses a child, it becomes MIN's turn.

The code also preserves the order of the children in `GAME_TREE`. Since the decision function updates the best move only when it finds a *strictly greater* value, the first action is retained when there is a tie.

Final result: Best move = C, Minimax value = 6.